GRACE: A Dynamic Coreset Selection Framework for Large Language Model Training Optimization

Technical Report

TIANHAO TANG, The Hong Kong University of Science and Technology, China
HAOYANG LI*, The Hong Kong Polytechnic University, China
LEI CHEN, The Hong Kong University of Science and Technology (Guangzhou), China and The Hong Kong University of Science and Technology, China

This technical report accompanies the GRACE paper and provides theorem proofs, implementation details, computation and graph costs, and additional empirical evidence.

1 Theoretical Proofs

Theorem and algorithm numbers, and equation numbers without a section prefix, refer to the main paper. Equations in this report use section-based numbering.

1.1 Proof of Theorem 4.1

Proof. For each sample X_i, we mask prompt, padding, and ignored-label positions to zero and vectorize the output logits and their gradients in a common padded space. We then define

$$\mathbf{Q}_i = \mathrm{vec}\left(\nabla_{\hat{\mathbf{O}}_i}\mathcal{L}(X_i, \boldsymbol{\theta})\right), \qquad \mathbf{J}_i = \frac{\partial \mathrm{vec}(\hat{\mathbf{O}}_i)}{\partial \boldsymbol{\theta}} \tag{1.1}$$

Theorem 4.1 uses the following local conditions:

$$\|\mathbf{J}_i\|_{\mathrm{op}} \le B_J, \qquad \|\mathbf{J}_i - \mathbf{J}_j\|_{\mathrm{op}} \le L_J\|\mathbf{X}_i - \mathbf{X}_j\|_2, \qquad \|\mathbf{Q}_i\|_2 \le B_Q \tag{1.2}$$

The first condition bounds the parameter-to-logit sensitivity, the second controls how this sensitivity varies with the input embeddings, and the third bounds the output-logit gradients in the considered local region.

By the chain rule, the vectorized parameter gradient of sample X_i is

$$\mathbf{g}_i = \mathrm{vec}\left(\nabla_{\boldsymbol{\theta}}\mathcal{L}(X_i, \boldsymbol{\theta})\right) = \mathbf{J}_i^\top \mathbf{Q}_i \tag{1.3}$$

For any pair of samples X_i and X_j,

$$\mathbf{g}_i - \mathbf{g}_j = \mathbf{J}_i^\top(\mathbf{Q}_i - \mathbf{Q}_j) + (\mathbf{J}_i - \mathbf{J}_j)^\top \mathbf{Q}_j \tag{1.4}$$

Applying the operator-norm bounds in Equation (1.2) gives

$$\begin{aligned}\|\mathbf{g}_i - \mathbf{g}_j\|_2 &\le \|\mathbf{J}_i\|_{\mathrm{op}}\|\mathbf{Q}_i - \mathbf{Q}_j\|_2 + \|\mathbf{J}_i - \mathbf{J}_j\|_{\mathrm{op}}\|\mathbf{Q}_j\|_2 \\ &\le B_J\|\mathbf{Q}_i - \mathbf{Q}_j\|_2 + L_J B_Q\|\mathbf{X}_i - \mathbf{X}_j\|_2\end{aligned} \tag{1.5}$$

*Corresponding author.

Authors' Contact Information: Tianhao Tang, Department of Computer Science and Engineering, The Hong Kong University of Science and Technology, Hong Kong SAR, China, ttangae@connect.ust.hk; Haoyang Li, Department of Computing, The Hong Kong Polytechnic University, Hong Kong SAR, China, haoyang-comp.li@polyu.edu.hk; Lei Chen, Data Science and Analytics Thrust, The Hong Kong University of Science and Technology (Guangzhou), Guangzhou, Guangdong, China, Department of Computer Science and Engineering and The Hong Kong University of Science and Technology, Hong Kong SAR, China, leichen@cse.ust.hk.

Since Equation (1.5) holds for every $X_j \in \mathcal{D}_{core}$, taking the minimum over X_j and summing over $X_i \in \mathcal{D}_{train}$ gives

$$\begin{aligned}&\sum_{X_i \in \mathcal{D}_{train}} \min_{X_j \in \mathcal{D}_{core}} \left\|\nabla_\theta \mathcal{L}(X_i, \theta) - \nabla_\theta \mathcal{L}(X_j, \theta)\right\|_2 \\ &\leq \sum_{X_i \in \mathcal{D}_{train}} \min_{X_j \in \mathcal{D}_{core}} \left[L_J B_Q \|\mathbf{X}_i - \mathbf{X}_j\|_2 + B_J \|\mathbf{Q}_i - \mathbf{Q}_j\|_2\right] \end{aligned} \tag{1.6}$$

Setting $c_1 = L_J B_Q$ and $c_2 = B_J$ yields both bounds in Theorem 4.1. □

1.2 Proof of Theorem 4.2

Proof. For the supervised fine-tuning loss, ignored positions contribute zero and the losses over $\mathcal{V}_i$ are averaged by N_i. The chain rule and the Jacobian bound in Equation (10) therefore give

$$\begin{aligned}\|\nabla_\theta \mathcal{L}(X_i, \theta)\|_2 &\leq B_J \left\|\mathrm{vec}\left(\nabla_{\hat{\mathbf{O}}_i} \mathcal{L}(X_i, \theta)\right)\right\|_2 \\ &= \frac{B_J}{N_i} \left(\sum_{s \in \mathcal{V}_i} \|\mathbf{q}_{i,s}\|_2^2\right)^{1/2} \\ &\leq \frac{B_J}{N_i} \sum_{s \in \mathcal{V}_i} \|\mathbf{q}_{i,s}\|_2 = B_J I(X_i). \end{aligned} \tag{1.7}$$

The reweighted gradient is unbiased because

$$\mathbb{E}_{i \sim p}\left[\frac{\nabla_\theta \mathcal{L}(X_i, \theta)}{n p_i}\right] = \frac{1}{n} \sum_{i=1}^{n} \nabla_\theta \mathcal{L}(X_i, \theta). \tag{1.8}$$

Its second moment therefore satisfies

$$\begin{aligned}\mathbb{E}_{i \sim p}\left[\left\|\frac{\nabla_\theta \mathcal{L}(X_i, \theta)}{n p_i}\right\|_2^2\right] &= \frac{1}{n^2} \sum_{i=1}^{n} \frac{\|\nabla_\theta \mathcal{L}(X_i, \theta)\|_2^2}{p_i} \\ &\leq \frac{B_J^2}{n^2} \sum_{i=1}^{n} \frac{I(X_i)^2}{p_i}, \end{aligned} \tag{1.9}$$

which proves Equation (11). Finally, Cauchy–Schwarz gives

$$\left(\sum_{i=1}^{n} I(X_i)\right)^2 \leq \left(\sum_{i=1}^{n} \frac{I(X_i)^2}{p_i}\right)\left(\sum_{i=1}^{n} p_i\right) = \sum_{i=1}^{n} \frac{I(X_i)^2}{p_i}. \tag{1.10}$$

Equality holds exactly when p_i is proportional to $I(X_i)$. Since the probabilities sum to one, the minimizing distribution is $p_i = I(X_i) / \sum_{j=1}^{n} I(X_j)$. □

1.3 Proof of Theorem 4.4

Proof. We reduce from Dominating Set, a classical NP-hard covering problem [1]. Let $G = (V, E)$ be an undirected graph with $n = |V|$ and budget b, and set $D = n$. For each vertex $v \in V$, construct an embedding $\mathbf{h}_v$ with one shared coordinate for every edge and $D - \deg(v)$ private coordinates used only by v. The edge coordinate for (u, v) equals one in both $\mathbf{h}_u$ and $\mathbf{h}_v$, every private coordinate equals one in its owner's embedding, and all other entries are zero. This construction has polynomial dimension and satisfies

$$\|\mathbf{h}_v\|_2^2 = D, \qquad \mathbf{h}_u^\top \mathbf{h}_v = \begin{cases} 1, & (u, v) \in E, \\ 0, & (u, v) \notin E \end{cases} \quad (u \neq v) \tag{1.11}$$

Consequently, the shifted-cosine kernel equals 1 for a vertex and itself, $\frac{1}{2} + \frac{1}{2D}$ for adjacent vertices, and $\frac{1}{2}$ for distinct non-adjacent vertices. Set $\lambda = 1$ and $\hat{I}(X_v) = 0$ for every candidate. For a size-b subset $S \subseteq V$, let $c(S)$ be the number of vertices in $V \setminus S$ adjacent to at least one vertex in S. The candidate-set objective then equals

$$R_{\mathcal{D}_S}(S) = b + \frac{n-b}{2} + \frac{c(S)}{2D} \tag{1.12}$$

It reaches $b + \frac{n-b}{2} + \frac{n-b}{2D}$ exactly when every vertex outside S has a neighbor in S, which is equivalent to S being a dominating set because every selected vertex covers itself. A dominating set of size at most b can be padded to size b, while a set with fewer than b vertices cannot reach this threshold. Thus, an exact polynomial-time solver for Equation (15) would decide Dominating Set in polynomial time, so the candidate-set coreset selection problem is NP-hard. □

1.4 Proof of Theorem 4.6

Proof. Fix a selection round. Write $D = \mathcal{D}_{train}$, $C = \mathcal{D}_S$, $N = |D|$, and $m = |C|$. Condition on the embeddings and non-negative warped importance scores on D before the uniform draw of C. For each $X_u \in D$ and $A \subseteq D$, define

$$f_u(A) = \begin{cases} \max_{X_j \in A} \kappa_{\cos}(X_u, X_j), & A \neq \emptyset, \\ 0, & A = \emptyset. \end{cases} \tag{1.13}$$

Because $\kappa_{\cos} \in [0, 1]$, each f_u is normalized and monotone. To establish diminishing returns, let $A \subseteq B \subseteq D$ and $X_x \in D \setminus B$. With $m_A = f_u(A)$ and $m_B = f_u(B)$, we have $0 \leq m_A \leq m_B$, and

$$\begin{aligned} f_u(A \cup \{X_x\}) - f_u(A) &= \max\{0, \kappa_{\cos}(X_u, X_x) - m_A\} \\ &\geq \max\{0, \kappa_{\cos}(X_u, X_x) - m_B\} \\ &= f_u(B \cup \{X_x\}) - f_u(B) \end{aligned} \tag{1.14}$$

Thus, f_u is submodular. The importance term $\hat{I}(A)$ is normalized, non-negative, and modular. Consequently, for $U \in \{D, C\}$, define

$$\begin{aligned} R_U(A) &= \sum_{X_u \in U} f_u(A), \\ F_U(A) &= \lambda R_U(A) + (1-\lambda)\hat{I}(A) \end{aligned} \tag{1.15}$$

Both are normalized, monotone, and submodular. They are the full-data comparator in Theorem 4.6 and the sampled objective evaluated by Algorithm 2, respectively:

$$F_D = RS_{\mathcal{D}_{train}}, \qquad F_C = RS_{\mathcal{D}_S}.$$

Let $\widehat{S}$ be the output of Algorithm 2 and let $c_b = 1 - (1 - 1/b)^b$. At greedy step t, submodularity implies that the largest available marginal gain is at least $1/b$ of the residual gap to the optimal size-b subset of C. Iterating this relation for b steps gives

$$F_C(\widehat{S}) \geq c_b \max_{A \subseteq C,\, |A| \leq b} F_C(A). \tag{1.16}$$

Because $C \subseteq D$, non-negativity gives $F_D(A) \geq F_C(A)$ for every $A \subseteq C$.

Let $S^\star$ maximize F_D over subsets of D of size at most b. If $F_D(S^\star) = 0$, the theorem follows from non-negativity. Otherwise, choose a nonempty optimal $S^\star$. For each $X_u \in D$, fix one representative

$a(u) \in S^\star$ attaining $f_u(S^\star)$. Since $S^\star \cap C$ is feasible for the maximization in Equation (1.16),

$$\begin{aligned}\mathbb{E}_C[R_C(S^\star \cap C)] &\geq \sum_{X_u \in D} \Pr\{X_u, a(u) \in C\} \kappa_{\cos}(X_u, a(u)) \\ &\geq \frac{m(m-1)}{N(N-1)} R_D(S^\star). \end{aligned} \tag{1.17}$$

The second inequality also covers $a(u) = X_u$, whose inclusion probability m/N is no smaller than the displayed joint probability. For the modular term, uniform sampling gives

$$\mathbb{E}_C[\hat{I}(S^\star \cap C)] = \frac{m}{N} \hat{I}(S^\star) \geq \frac{m(m-1)}{N(N-1)} \hat{I}(S^\star). \tag{1.18}$$

Combining Equations (1.16)–(1.18) yields

$$\begin{aligned}\mathbb{E}_C[F_D(\widehat{S})] &\geq \mathbb{E}_C[F_C(\widehat{S})] \\ &\geq c_b \, \mathbb{E}_C[F_C(S^\star \cap C)] \\ &\geq c_b \frac{m(m-1)}{N(N-1)} F_D(S^\star) \\ &\geq \left(1 - \frac{1}{e} - \varepsilon_m\right) F_D(S^\star), \end{aligned} \tag{1.19}$$

where the final inequality uses $c_b \geq 1 - 1/e$ and the definition of ε_m in Theorem 4.6. □

1.5 Proof of Theorem 4.7

Proof. Define

$$e_i^{\text{old}} = I(X_i) - I_t^\star(X_i), \qquad e_i^{\text{new}} = I^{new}(X_i) - I_t^\star(X_i)$$

Substituting Equation (20) into $e_i^{\text{new}} = I^{new}(X_i) - I_t^\star(X_i)$ yields

$$\begin{aligned} e_i^{\text{new}} &= \frac{1}{2} I(X_i) + \frac{1}{2} \sum_{j \in \mathcal{N}_i^\star} \alpha_{ij} I^{new}(X_j) - I_t^\star(X_i) \\ &= \frac{1}{2} e_i^{\text{old}} + \frac{1}{2} \sum_{j \in \mathcal{N}_i^\star} \alpha_{ij} \Big(I^{new}(X_j) - I_t^\star(X_i) \Big) \end{aligned}$$

Add and subtract $I_t^\star(X_j)$ inside the summation:

$$\begin{aligned} e_i^{\text{new}} = \frac{1}{2} e_i^{\text{old}} &+ \frac{1}{2} \sum_{j \in \mathcal{N}_i^\star} \alpha_{ij} \Big(I^{new}(X_j) - I_t^\star(X_j) \Big) \\ &+ \frac{1}{2} \sum_{j \in \mathcal{N}_i^\star} \alpha_{ij} \Big(I_t^\star(X_j) - I_t^\star(X_i) \Big) \end{aligned}$$

Taking absolute values and applying the triangle inequality gives

$$\begin{aligned} \left| e_i^{\text{new}} \right| \leq \frac{1}{2} \left| e_i^{\text{old}} \right| &+ \frac{1}{2} \sum_{j \in \mathcal{N}_i^\star} \alpha_{ij} \left| I^{new}(X_j) - I_t^\star(X_j) \right| \\ &+ \frac{1}{2} \sum_{j \in \mathcal{N}_i^\star} \alpha_{ij} \left| I_t^\star(X_j) - I_t^\star(X_i) \right| \end{aligned}$$

Using the propagated-source error bound and the local smoothness assumption, we obtain

$$\left|e_i^{\text{new}}\right| \le \frac{1}{2}\left|e_i^{\text{old}}\right| + \frac{1}{2}\sum_{j\in\mathcal{N}_i^\star}\alpha_{ij}\nu_j + \frac{1}{2}L_I\sum_{j\in\mathcal{N}_i^\star}\alpha_{ij}\left\|\bar{\mathbf{H}}_j - \bar{\mathbf{H}}_i\right\|$$

Since $\sum_{j\in\mathcal{N}_i^\star}\alpha_{ij} = 1$ and $\|\bar{\mathbf{H}}_j - \bar{\mathbf{H}}_i\| \le r_i$ for all $j \in \mathcal{N}_i^\star$, it follows that

$$\left|e_i^{\text{new}}\right| \le \frac{1}{2}\left|e_i^{\text{old}}\right| + \frac{1}{2}\sum_{j\in\mathcal{N}_i^\star}\alpha_{ij}\nu_j + \frac{1}{2}L_I r_i$$

Substituting back $e_i^{\text{new}} = I^{new}(X_i) - I_t^\star(X_i)$ and $e_i^{\text{old}} = I(X_i) - I_t^\star(X_i)$ proves the theorem. □

2 Implementation Details

2.1 GRACE Hyperparameters

Table 1 consolidates the GRACE hyperparameters used in the implementation and sensitivity study.

Table 1. GRACE hyperparameters and their configurations or determination rules.

Parameter	Configuration or determination
n_S	Per-round candidate set: full training set (BIRD, Spider); a uniformly sampled half of the training set (Jellyfish); a uniformly sampled quarter of the training set (MathInstruct).
n_{check}	At most 500 samples per update check; each check uses $\min\{500, \lvert\mathcal{D}_{\text{hist}}\rvert\}$ samples.
η	Within-round selection ratio $b/\lvert\mathcal{D}_S\rvert$ under each reported training budget. Budget study: {2%, 5%, 10%, 20%, 30%}
λ	Main: setting-specific selection with 0.1 increments. Sensitivity: $\{0, 0.2, 0.4, 0.6, 0.8, 1\}$
δ	Main: 0.2. Sensitivity: $\{0, 0.05, 0.10, 0.15, 0.20, 0.25, 0.30, 0.35\}$
$t_c/\mathcal{T}_e$	Main: 1 (BIRD, Spider), 0.5 (Jellyfish), 0.25 (MathInstruct). Sensitivity: $\{0.125, 0.25, 0.5, 1\}$
k	10, with graph-kernel denominator 100
k_{recal}	Samples recomputed per event: $0.25\lvert\mathcal{D}_{\text{train}}\rvert$ (BIRD, Spider) and $0.10\lvert\mathcal{D}_{\text{train}}\rvert$ (Jellyfish, MathInstruct).
C, γ	Main: $C = 5$, $\gamma = 1$. Sensitivity: $C \in \{3, 4, 5, 6\}$, $\gamma \in \{0.25, 0.5, 1, 2\}$
α, β	$\alpha = 1 + 5\,\text{Mean}(\tilde{I})\eta$, $\beta = 5 - \alpha$
λ_c	0.99
$\delta_h, \tau_{\text{aff}}$	$\delta_h = 0$ and $\tau_{\text{aff}} = 0.5$
Propagation constants	0.1 and 0.5

2.2 Baseline Implementation Details

LESS and NICE. Both methods use the same 100-example reference-set construction. We draw a fixed set of 100 distinct reference examples from the current benchmark's training set and retain them in the candidate pool. When a reference example is matched against its own candidate row, we mask the self-similarity entry. For MathInstruct, matching groups examples by source dataset, while Jellyfish uses four task groups. Reference similarities are averaged within each group, and the maximum group score is used across groups. For the other benchmarks, all reference examples form one group and their similarities are averaged.

STAFF. For the main backbones, STAFF pairs Phi-2 with Phi-1.5, Llama2-7b with a Sheared-LLaMA-2.7B, a third-party pruned and compressed model derived from Llama2-7b, and Qwen2.5-7b with Qwen2.5-3b. For the additional-backbone results, Qwen3-4b uses Qwen3-1.7b as its proxy, and Llama3.1-8b uses Llama3.2-3b. Each proxy is LoRA fine-tuned on the complete training pool with

all linear layers targeted, rank 8, scale factor 16, and dropout 0. STAFF then computes per-example effort using the fine-tuned proxy and the target-backbone scorer before target-model fine-tuning.
MATES. For every benchmark and target backbone, MATES uses the *bert-base-uncased* backbone with a single-output regression head and *num_labels*=1. The auxiliary model is trained for 5 epochs with a learning rate of 5×10^{-5} and weight decay of 0.01. The remaining procedure follows the original MATES protocol [2].
Configuration selection. Each method starts from its original-paper or released-code configuration. To keep tuning effort comparable, setting-specific adjustment is restricted to a small set of method-relevant parameters, including λ for GRACE, K for TAGCOS, and sample or cluster sizes for kMQ. All remaining settings retain their published or released values.

3 Efficiency and Graph Costs

3.1 Compute-Accounting Details

We measure end-to-end cost on Jellyfish with Llama2-7b, a 10% coreset budget, and one 80-GiB NVIDIA A100 accelerator. Target training counts the optimizer work used to fine-tune the target model. Figure 1 separately reports warm-up, offline preparation, target training, selection and setup, online scoring or recomputation, graph construction and maintenance, and other run costs. All components enter the end-to-end total; elapsed hours equal single-GPU job hours.

For comparison, full-data LoRA takes 9.259 single-GPU hours under the same Jellyfish and Llama2-7b configuration.

Table 2. Processed-example counts and peak accelerator allocation for Figure 1. Target-model and auxiliary-model work are reported separately. Each count sums the examples processed across the corresponding forward or backward stages rather than micro-batch model calls.

Method	Target fwd.	Target bwd.	Aux. fwd.	Aux. bwd.	Peak GiB
	Processed examples (10^3)				
Full-data LoRA	403.200	403.200	0	0	58.42
Random	40.704	40.704	0	0	56.38
Middle Perplexity	215.904	81.408	0	0	55.31
Facility Location	215.904	81.408	0	0	56.36
DavIR	350.400	81.408	0	0	52.49
Long4Align	40.704	40.704	0	0	52.08
GradN	215.904	81.408	0	0	56.01
Least Confidence	215.904	81.408	0	0	52.08
TAGCOS	215.904	215.904	0	0	56.50
LESS	484.896	484.896	0	0	50.27
NICE	490.896	490.896	0	0	53.38
STAFF	175.200	309.696	537.696	672.192	57.62
Fix Interval Random	40.704	40.704	0	0	55.66
DynamicMP	444.192	40.704	0	0	52.38
DynamicFL	444.192	40.704	0	0	56.34
kMQ	67.667	40.704	134.496	0	55.44
LEAD	484.896	81.408	0	0	58.10
MATES	2060.704	60.704	500.992	90.000	63.74
GRACE	293.149	81.408	0	0	55.40

Additional NICE generation cost. NICE also generates 6,000 response sequences through 300 autoregressive generation invocations. These invocations are fully included in the elapsed time reported in Figure 1. We report them separately because they perform variable-length autoregressive decoding, whereas the table columns count processed examples in fixed forward and backward stages.

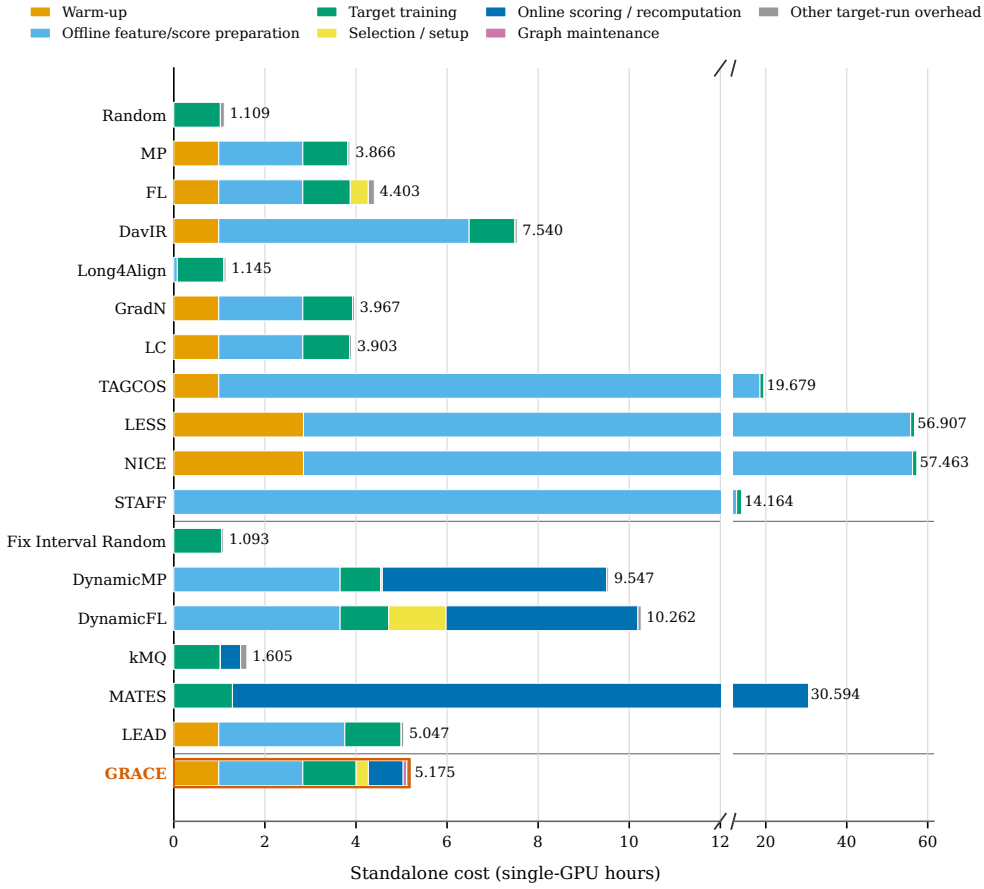


Fig. 1. Component-level single-GPU hours on Jellyfish with Llama2-7b at a 10% training budget. The stacked bars separate warm-up, offline preparation, target training, selection and setup, online scoring or recomputation, graph construction and maintenance, and other target-run overhead. The graph component includes initial construction and subsequent updates.

3.2 Graph Storage, Construction, and Update Scaling

Table 3 reports the serialized state and CPU memory footprint of the full graph. All datasets use Llama2-7b embeddings (4096-dimensional, float32) and $k = 10$. The graph stores embeddings, neighbor links, and auxiliary arrays. Its serialized size per example remains nearly constant as the dataset grows, while the CPU graph memory footprint scales from 0.579 GiB on Spider to 10.181 GiB on MathInstruct. Initial exact k-NN graph construction takes 2 seconds on Spider, 14 seconds on Jellyfish, and 44 seconds on MathInstruct. Initial construction uses exact FAISS k-NN search, while the LSH index is used only to retrieve candidates for incremental local repair before exact distance refinement. All construction and update timing measurements use one 80-GiB NVIDIA A100 accelerator.

Table 4 reports median per-event update times for Spider, BIRD, and Jellyfish with Llama2-7b and Qwen2.5-7b. The displayed update ratio is $k_{\text{recal}}/|\mathcal{D}_{\text{train}}|$, the fraction of the full training set

Table 3. Serialized graph storage, initial construction, and CPU graph memory footprint under a fixed Llama2-7b, 4096-dimensional float32 representation with $k = 10$. Construction uses one 80-GiB NVIDIA A100 accelerator.

Dataset	Examples	Stored links	Total MiB	KiB/ example	Construction (s)	CPU graph memory (GiB)
Spider	8,659	84,630	136.79	16.177	2	0.579
Jellyfish	134,496	1,263,216	2,128.06	16.202	14	5.515
MathInstruct	261,354	2,351,398	4,127.66	16.172	44	10.181

recomputed at each event. In larger Jellyfish runs, recomputation dominates and graph updating remains smaller.

Table 4. Median per-event update times across datasets and backbones using one 80-GiB NVIDIA A100 accelerator.

Dataset	Model	Examples	Update ratio	Recomputation (s)	Graph update (s)	Single update (s)
Spider	Llama2-7b	8,659	0.25	11.50	19.87	31.37
Spider	Qwen2.5-7b	8,659	0.25	35.02	21.57	56.59
BIRD	Llama2-7b	9,428	0.25	21.14	11.55	32.69
BIRD	Qwen2.5-7b	9,428	0.25	45.32	23.66	68.98
Jellyfish	Llama2-7b	134,496	0.10	559.64	47.08	606.72
Jellyfish	Qwen2.5-7b	134,496	0.10	562.31	33.80	596.11

4 Additional Empirical Evidence

4.1 Evidence for Non-uniform Model Drift Across Samples

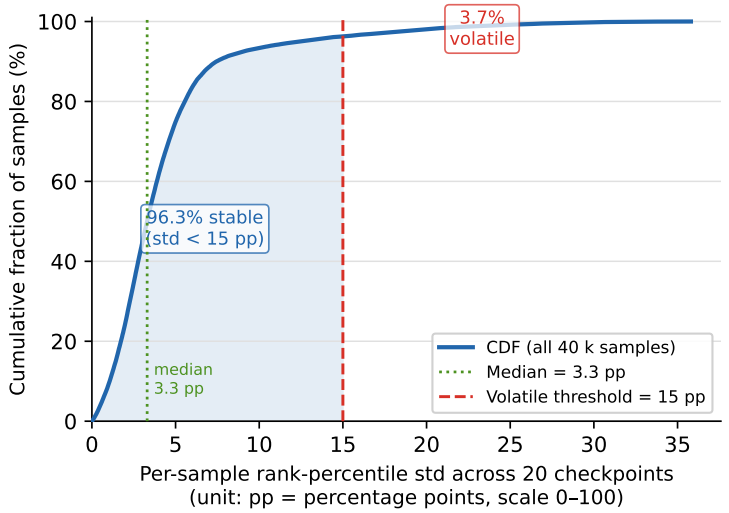
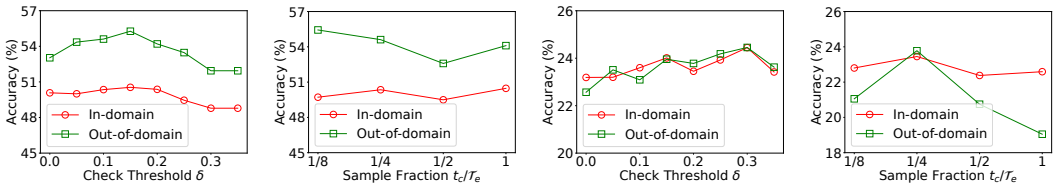


Fig. 2. CDF of per-sample rank volatility during full-data LoRA fine-tuning of Llama2-7b on MathInstruct

We examine the non-uniform model drift discussed in Section 4.3 of the main paper by measuring changes in sample difficulty rankings. We use Llama2-7b with full-data LoRA fine-tuning on MathInstruct and measure importance scores for 40,000 training samples every 100 training steps from step 100 to step 2000. At each checkpoint, we convert the importance scores of all samples into percentile ranks, where 100 denotes the hardest ones. For each sample, we then compute the standard deviation of its rank percentile across the 20 checkpoints, which measures how much its relative difficulty changes over time.

Figure 2 plots the CDF of this per-sample rank volatility. The distribution is strongly concentrated near low volatility values: the median is 3.3 percentage points, and 96.3% of samples have volatility below 15 percentage points, while only 3.7% form a high-volatility tail. These results support our observation that, over short training intervals, most samples remain relatively stable, while only a small subset changes substantially.

4.2 Checking Threshold and Interval



(a) δ on Phi-2 (b) $t_c/\mathcal{T}_e$ on Phi-2 (c) δ on Llama2-7b (d) $t_c/\mathcal{T}_e$ on Llama2-7b

Fig. 3. Sensitivity to the checking threshold δ and check-interval fraction $t_c/\mathcal{T}_e$ on MathInstruct.

Checking threshold δ. Figures 3a and 3c vary δ in Equation (17) from 0 to 0.35 in 0.05 increments. When δ is close to 0, update checks trigger frequently and approach non-adaptive updating. At $\delta \geq 0.3$, most updates are suppressed and the behavior approaches static selection. Both models perform best at moderate thresholds, balancing update frequency and suppression.

Checking interval t_c. Let $\mathcal{T}_e$ be the optimizer steps per epoch and t_c the steps between consecutive update checks, so $t_c/\mathcal{T}_e$ is the check-interval fraction. Figures 3b and 3d compare 0.125, 0.25, 0.5, and 1. Across both models, in-domain performance is relatively stable, while out-of-domain accuracy is more sensitive to the interval. Moderate fractions provide the best overall balance and outperform both the most frequent and coarsest settings.

4.3 Additional Training-Budget Results

Figure 4 reports the Phi-2 budget results corresponding to the Llama2-7b results in Section 5.4 of the main paper.

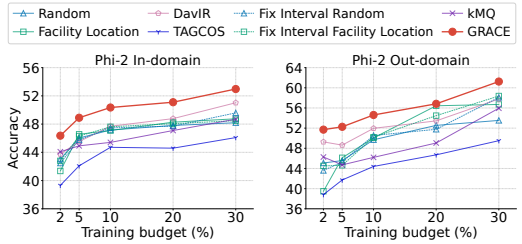


Fig. 4. Experiment results under different training budgets on MathInstruct with Phi-2.

References

[1] Dorit S. Hochbaum. 1996. *Approximating covering and packing problems: set cover, vertex cover, independent set, and related problems.* PWS Publishing Co., USA, 94–143.

[2] Zichun Yu, Spandan Das, and Chenyan Xiong. 2024. MATES: Model-Aware Data Selection for Efficient Pretraining with Data Influence Models. In *Advances in Neural Information Processing Systems*, A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang (Eds.), Vol. 37. Curran Associates, Inc., 108735–108759. doi:10.52202/079017-3453